PlainCNN50 vs DenseNet201

Performance Comparison on CIFAR-100

Michele Cirillo
miccirillo2003@gmail.com

Deep Learning Research

February 15, 2026

Study Objective

PlainCNN50

- 50 convolutional layers
- Sequential architecture
- Simple but deep

DenseNet201

- Dense blocks with skip connections
- Feature reuse
- Improved gradient flow

Dataset: CIFAR-100

Dataset Characteristics

- 100 classes (fine-grained)
- 50,000 training images
- 10,000 test images
- Size: 32×32×3 pixels
- Challenge: high intra-class similarity

Data Augmentation

- Random Horizontal Flip
- Random Crop (32×32)
- Normalization [0,1]

Complexity

CIFAR-100 is significantly more challenging than CIFAR-10 (10 classes), requiring more sophisticated architectures.

PlainCNN50: Architecture

Structure

1. **Input:** 32×32×3
2. **50 Conv Layers:**
   - 3×3 kernels
   - BatchNorm + ReLU
   - Channel growth: 32→512
3. **Global Average Pooling**
4. **FC Layer:** 100 classes

Characteristics

- Direct sequential flow
- Progressive feature growth
- Each layer processes previous output
- Problem: Vanishing gradient

Key Points

- No skip connections
- Information loss through layers
- Difficult gradient propagation

DenseNet201: Architecture

Dense Blocks

- Each layer receives input from **all** previous layers
- Concatenation instead of summation
- Growth rate: k=32
- 201 layers total

Transition Layers

- Batch Normalization
- 1×1 Convolution
- 2×2 Average Pooling
- Dimensionality reduction

Advantages

- Feature reuse
- Direct gradient flow
- Fewer parameters
- Implicit regularization

Key Benefits

- Skip connections preserve information
- Better gradient propagation
- More efficient parameter usage

Skip Connections

What Skip Connections Do

- Create **direct pathways** that bypass intermediate layers
- Allow information to flow from earlier layers directly to deeper layers
- In DenseNet: each layer receives inputs from **all previous layers**
- Features are **concatenated**, preserving all information

Why Skip Connections Are Useful

- **Solve vanishing gradient problem:** Gradients can flow directly backward through shortcuts
- **Feature reuse:** Low-level features remain accessible throughout the network
- **Enable deeper networks:** Networks with 100+ layers can be trained effectively

Training Configuration

Hyperparameters

- **Batch Size:** 128
- **Epochs:** 50
- **Optimizer:** Adam
- **Learning Rate:** 0.001
- **Loss:** CrossEntropyLoss

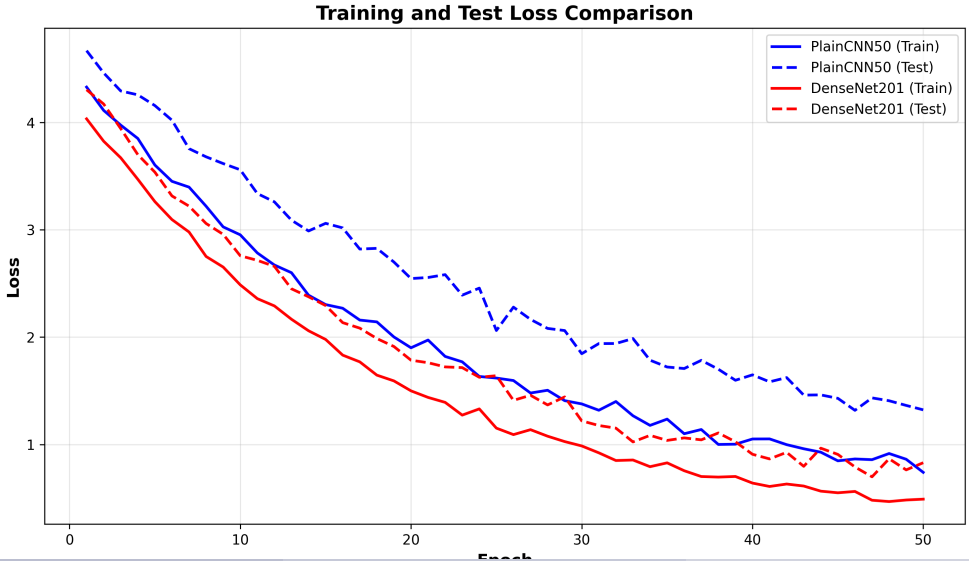
Hardware & Framework

- GPU: CUDA enabled
- Framework: PyTorch
- Libraries: torchvision, torchmetrics, sklearn

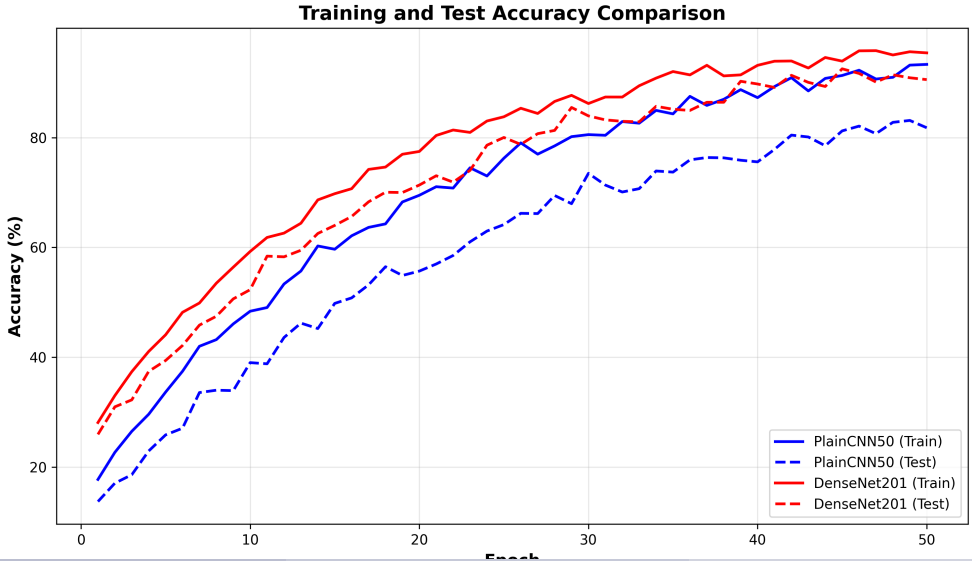
Evaluation Metrics

- Training & Test Loss
- Training & Test Accuracy
- F1 Score (weighted)
- EER (Equal Error Rate)

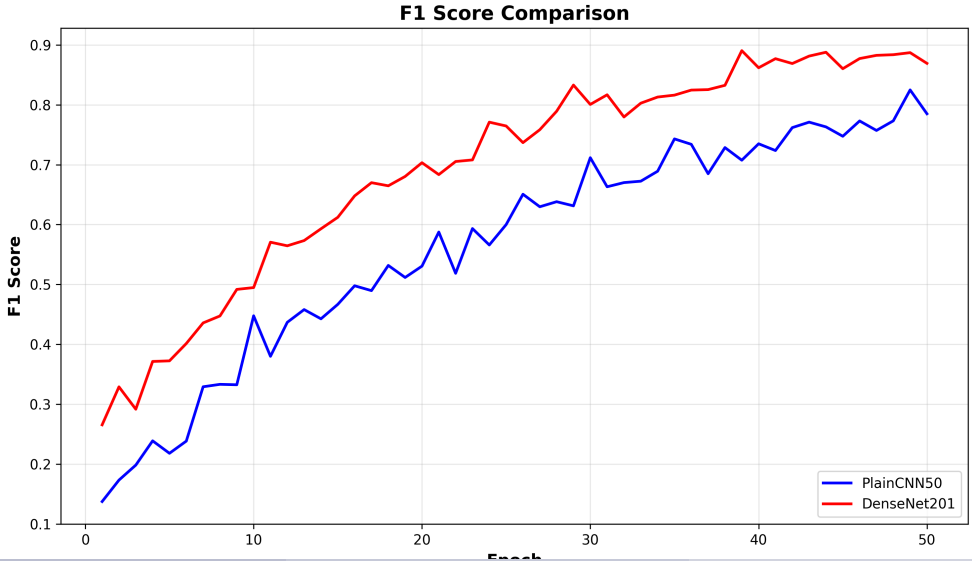
Loss Evolution



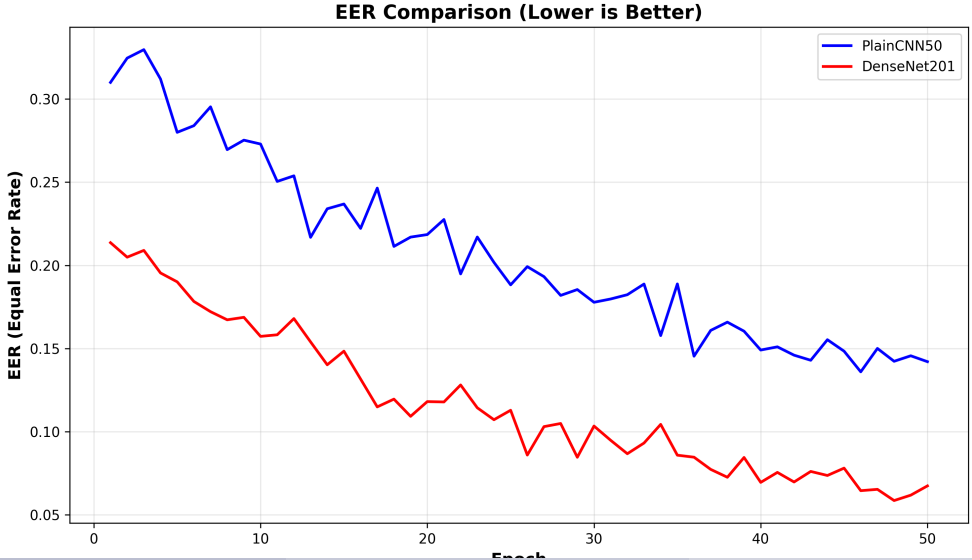
Accuracy Evolution



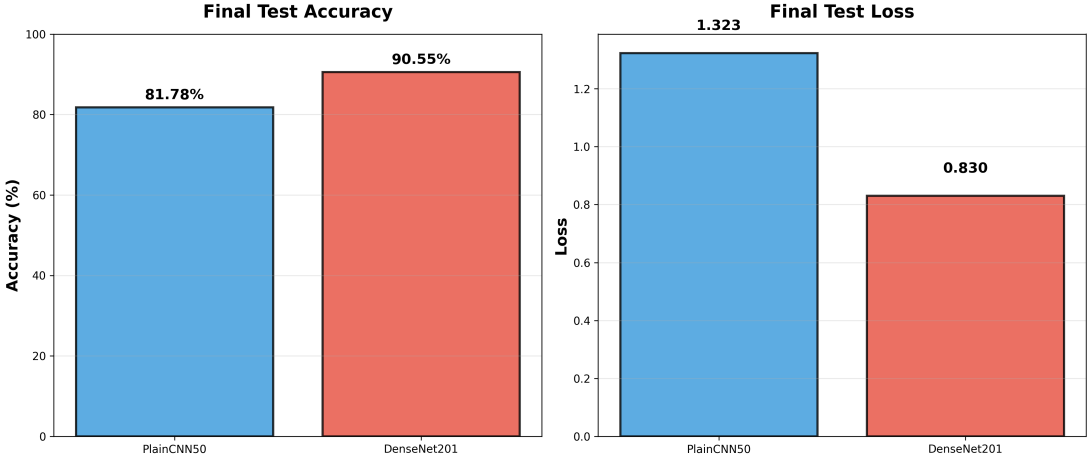
F1 Score Evolution



EER Evolution

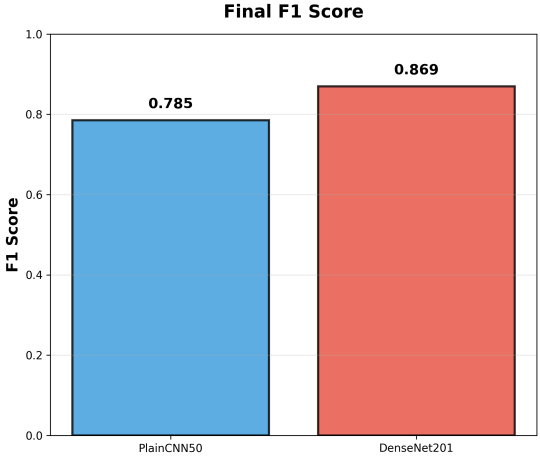


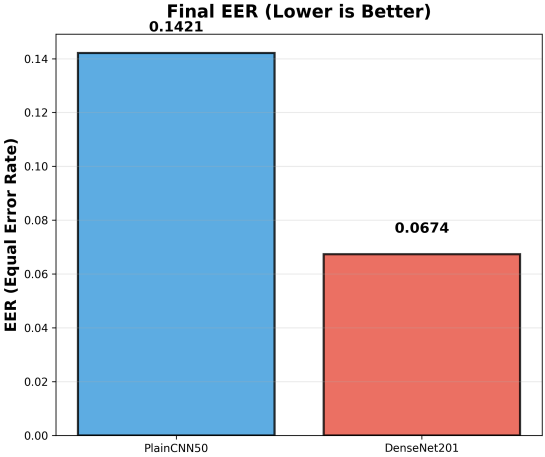
Final Results - Accuracy & Loss



- DenseNet201 achieves **8.76% higher accuracy**
- DenseNet201 has **37.2% lower loss**

Final Results - F1 Score & EER





- DenseNet201 shows **better F1 score** (0.869 vs 0.785)
- DenseNet201 has **significantly lower EER** (0.0674 vs 0.1421)

Final Metrics Comparison

Model	Accuracy	Loss	F1 Score	EER
PlainCNN50	81.78%	1.323	0.785	0.1421
DenseNet201	**90.55%**	**0.830**	**0.869**	**0.0674**
Improvement	**+8.76%**	**-37.2%**	**+0.084**	**-0.0747**

Key Findings

- DenseNet201 outperforms PlainCNN50 across **all metrics**
- Significant improvement in accuracy (+8.76%)
- Better loss convergence (-37.2%)
- Superior F1 score and lower error rate

Results Analysis

DenseNet201 Wins Due To:

- **Skip connections**: Improved gradient flow
- **Feature reuse**: Preserved information
- **Efficient parameters**: Less overfitting
- **Regularization**: Concatenation = implicit dropout

PlainCNN50 Limitations:

- Vanishing gradient in deep networks
- Information lost layer by layer
- More parameters but less effective
- Slower convergence

DenseNet201: Superior across all metrics

Conclusions

Main Results

1. **DenseNet201 superior**: 90.55% vs 81.78% accuracy
2. **Skip connections crucial**: Improve gradient flow
3. **Parameter efficiency**: Fewer parameters, better results
4. **Training stability**: DenseNet converges faster
5. **Consistent superiority**: Better on all metrics (Acc, Loss, F1, EER)

When to Use DenseNet

- Complex tasks (100+ classes)
- Limited resources
- Need for fast convergence

When to Use PlainCNN

- Simple tasks
- Few layers sufficient
- Interpretability priority

References

- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). *Densely connected convolutional networks*. CVPR 2017.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. CVPR 2016.
- Krizhevsky, A., & Hinton, G. (2009). *Learning multiple layers of features from tiny images*. Technical report, University of Toronto.

Thank you for your attention!